Modéliser un trafic attendu

Régression linéaire et analyse des écarts de trafic réseau

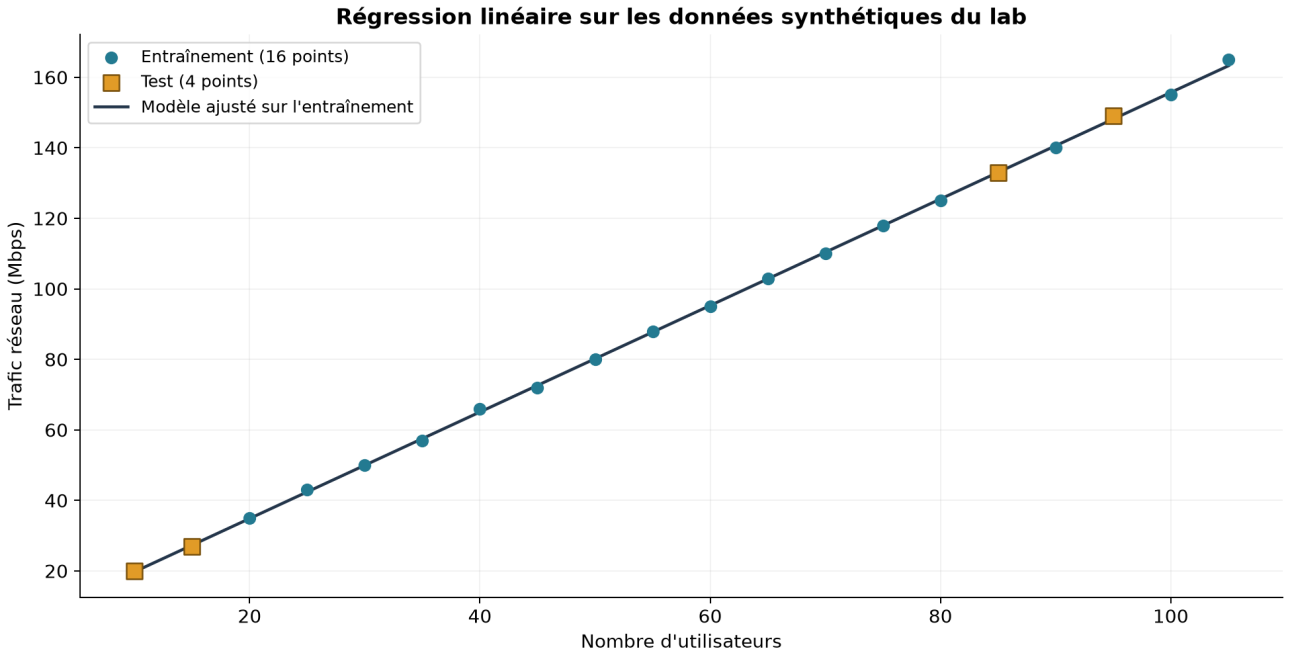
Otmane Arajdal
5e année de génie informatique, option cybersécurité

Dossier de présentation GitHub - reconstruction vérifiée le 20 octobre 2026.

La question de départ

Comment construire une première référence du trafic réseau à partir du nombre d'utilisateurs, puis interpréter un écart ? Ce lab m'a permis de relier un modèle simple à une démarche d'investigation, en gardant une distinction entre signal statistique et incident confirmé.

Jeu pédagogique	Entraînement	Test
20 couples synthétiques	16 observations	4 observations



Les couleurs distinguent les données utilisées pour ajuster la droite de celles utilisées pour l'évaluer. Graphe produit par la réexécution.

Démarche suivie

Définir X = nombre d'utilisateurs et y = trafic en Mbps ; conserver 20 % des points pour le test avec random_state=42 ; ajuster une régression linéaire ; calculer les erreurs ; examiner un trafic simulé par rapport à la prévision.

Périmètre. Les valeurs sont saisies dans le support pédagogique. Aucune capture de trafic réel, attaque ou étiquette d'incident n'est utilisée. Le notebook autonome et le script du dépôt rendent cette démarche reproductible.

Comprendre les résultats

Une droite explicable

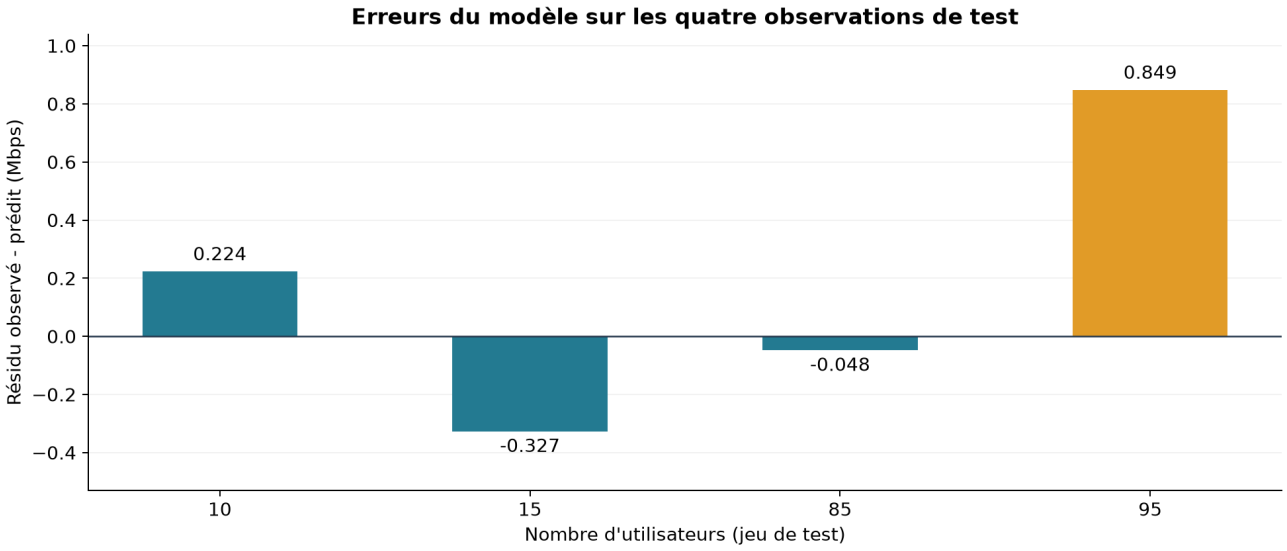
Trafic prédit = 4,672900 + 1,510300 × utilisateurs

Dans ce modèle, un utilisateur supplémentaire est associé à environ 1,51 Mbps dans la prévision. Cette pente ne prouve pas une causalité. L'intercept extrapole à zéro utilisateur, hors des données : ce n'est pas une mesure du trafic de fond.

MAE sur le test	MSE sur le test	R² sur le test
0,362125 Mbps	0,219965 (Mbps)²	0,9999369671

Quatre observations gardées hors apprentissage

Utilisateurs	Observé (Mbps)	Prédit (Mbps)	Résidu (Mbps)
10	20	19,775901	+0,224099
95	149	148,151410	+0,848590
85	133	133,048409	-0,048409
15	27	27,327402	-0,327402



Résidu = observé - prédit. Le plus grand écart absolu du test vaut environ 0,8486 Mbps, pour 95 utilisateurs.

Ce que les métriques permettent de dire

La faible MAE montre que le modèle reproduit bien les quatre points de ce découpage. Le jeu est petit et presque linéaire : un R² proche de 1 n'indique pas une performance de détection de cyberattaques. Les points à 10 et 15 utilisateurs sont même sous le minimum d'entraînement, qui vaut 20.

Une comparaison supplémentaire à une constante égale à la moyenne d'entraînement est fournie dans le notebook et le JSON. Cet ajout donne un repère simple, sans élargir la validité des résultats au trafic réel.

Du résidu à l’investigation

Le scénario du lab

Pour **110 utilisateurs**, le trafic attendu est de **170,805912 Mbps**. Le support simule une observation à **220 Mbps** : le résidu vaut **+49,194088 Mbps**. La règle **|résidu| > 40 Mbps** demande donc une investigation.

L’entrée 110 dépasse le maximum observé de 105 : c’est une légère extrapolation. Le seuil de 40 est arbitraire, sans calibration sur des événements opérationnels.

Trafic simulé	Résidu (Mbps)	Règle déclenchée
160 Mbps (ajout)	-10,805912	Non
180 Mbps (ajout)	+9,194088	Non
220 Mbps (support)	+49,194088	Oui
120 Mbps (ajout)	-50,805912	Oui

Les trois variantes sont ajoutées lors de la reproduction. Une baisse importante déclenche aussi la règle absolue. Un écart exactement égal à 40 ne la déclenche pas.

Rechercher la cause avant de conclure

Une mise à jour, une sauvegarde, un transfert volumineux ou un défaut de mesure peuvent produire un écart. Il faudrait rapprocher le signal des flux, protocoles, applications, authentifications et tâches planifiées. Le contexte horaire se lit par rapport aux habitudes du système.

Limites et suite du travail

Le jeu ne contient ni historique daté ni étiquettes d’attaque. Aucun rappel d’incidents, taux de faux positifs ou intervalle de confiance n’est validé. Le split aléatoire du lab ne remplace pas une évaluation chronologique sur un réseau réel.

Pour poursuivre, je collecterais un historique avec une fenêtre de mesure définie, comparerais plusieurs références et calibrerais un seuil sur une validation distincte. Ces prolongements restent à expérimenter.

Ce que je retiens

Le principal apprentissage est de savoir expliquer les étapes : le modèle estime, le résidu mesure l’écart et l’analyste en recherche la cause. Le dépôt conserve les données, le code, les résultats et leur provenance pour que cette lecture puisse être vérifiée.

Sources et reproduction

Support fourni : « Régression linéaire appliquée à la cybersécurité », 11 pages, données et captures Colab. Notebook recréé et exécuté ; deux captures de résultats extraites intactes. Les versions et sorties détaillées figurent dans le dépôt.

Références : scikit-learn - LinearRegression ; train_test_split ; métriques de régression.